

Report Pratica

Python & AI per la Cyber Security

Obiettivo della pratica

L'obiettivo di questa esercitazione era imparare i fondamenti dell'interazione con i modelli linguistici (LLM) tramite Python, applicati al contesto della Cyber Security. In particolare ho dovuto:

- Configurare l'ambiente Python con l'API di Gemini (virtual environment, libreria google-genai, pandas)
- Creare uno script "Hello Gemini" per inviare un prompt e stampare la risposta
- Creare un Chatbot Security Assistant interattivo, con un system prompt dedicato e un ciclo di conversazione

Script "Hello Gemini" (hello_gemini.py)

Per prima cosa ho scritto uno script base per verificare la comunicazione con il modello Gemini. Lo script carica la chiave API dal file .env, crea il client e invia un prompt che chiede al modello di confermare che l'ambiente Python sia configurato correttamente per operazioni di Cyber Security:



```
hello_gemini.py > ...
1  from google import genai
2  import os
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  api_key = os.environ["GEMINI_API_KEY"]
8  client = genai.Client(api_key=api_key)
9
10 prompt = "Ciao Gemini! Conferma che l'ambiente Python è configurato correttamente per le operazioni di Cyber Security."
11
12 response = client.models.generate_content(
13     model="gemini-3.6-flash",
14     #contents="ciao, come stai?",
15     contents=prompt
16 )
17
18 print("__ Risposta da Gemini __")
19 print(response.text)
```

Esecuzione di hello_gemini.py e verifica dell'ambiente

Eseguendo lo script, Gemini ha risposto spiegando di non poter accedere direttamente al mio computer, ma proponendo uno script Python (check_env.py) per verificare in autonomia quali librerie di Cyber Security fossero già installate nell'ambiente:

```
(venv ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python hello_gemini.py
(venv ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python hello_gemini.py
Direct use of automatic function calling (AFC) in Models.generate content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Risposta da Gemini
Ciao! Essendo un'intelligenza artificiale in remoto, **non posso accedere direttamente al tuo computer** o al tuo terminale per eseguire comandi e verificare lo stato del tuo sistema. Tuttavia, posso aiutarti a **verificarlo tu stesso in pochi secondi**.

Ecco uno script Python che ho preparato per te. Copia ed esegui questo codice nel tuo ambiente Python per verificare se le librerie fondamentali per la Cyber Security sono installate e pronte all'uso.

### Script di verifica dell'ambiente

Crea un file chiamato 'check_env.py' e incolla questo codice:

```python
import importlib
import sys

print(f"=== VERIFICA AMBIENTE PYTHON ===")
print(f"Versione Python: {sys.version.split()[0]}")
if sys.version_info < (3, 8):
 print(
 "[!] ATTENZIONE: Si consiglia Python 3.8 o superiore per le operazioni di sicurezza."
)
else:
 print("[✓] Versione Python OK.\n")

Lista delle principali librerie usate in Cyber Security
cyber_libraries = {
 # Networking & Packet Crafting
 "scapy": "Scapy (Manipolazione pacchetti di rete)",
 "nmap": "python-nmap (Wrapper per Nmap)",
 # Web Security & Scraping
 "requests": "Requests (Richieste HTTP)",
 "bs4": "BeautifulSoup4 (Parsing HTML/Web Scraping)",
 "urllib3": "Urllib3 (Client HTTP avanzato)",
 # Cryptography
 "cryptography": "Cryptography (Cifratura e certificati)",
 "Crypto": "PyCryptodome (Algoritmi crittografici)",
 # Exploit Dev & Reverse Engineering
 "pwn": "Pwntools (CTF & Exploit Development)",
 "pefile": "Pefile (Analisi di file eseguibili Windows PE)",
}
```

Lo script suggerito da Gemini definisce un dizionario con le principali librerie utilizzate in ambito Cyber Security (networking e packet crafting, web security e scraping, crittografia, exploit development) e ne verifica la presenza tramite importlib:

```
(venv ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python hello_gemini.py
"scapy": "Scapy (Manipolazione pacchetti di rete)",
"nmap": "python-nmap (Wrapper per Nmap)",
"paramiko": "Paramiko (Connessioni SSH/SFTP)",
"impacket": "Impacket (Protocolli di rete / Active Directory)",
Web Security & Scraping
"requests": "Requests (Richieste HTTP)",
"bs4": "BeautifulSoup4 (Parsing HTML/Web Scraping)",
"urllib3": "Urllib3 (Client HTTP avanzato)",
Cryptography
"cryptography": "Cryptography (Cifratura e certificati)",
"Crypto": "PyCryptodome (Algoritmi crittografici)",
Exploit Dev & Reverse Engineering
"pwn": "Pwntools (CTF & Exploit Development)",
"pefile": "Pefile (Analisi di file eseguibili Windows PE)",

print("=== VERIFICA LIBRERIE CYBER SECURITY ===")
missing_libs = []

for lib, desc in cyber_libraries.items():
 try:
 importlib.import_module(lib)
 print(f"[✓] INSTALLATO: {desc}")
 except ImportError:
 print(f"[X] MANCANTE: {desc}")
 # Mappatura dei nomi pip per l'installazione
 pip_name = lib
 if lib == "bs4":
 pip_name = "beautifulsoup4"
 elif lib == "Crypto":
 pip_name = "pycryptodome"
 elif lib == "pwn":
 pip_name = "pwntools"
 elif lib == "nmap":
```

Al termine della verifica, lo script stampa un esito riassuntivo (librerie installate o mancanti), suggerisce il comando per installare eventuali pacchetti mancanti e riporta alcune best practice per l'ambiente di Cyber Security, come l'uso di un virtual environment dedicato:

```
(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python hello_gemini.py

print("\n=== ESITO ===")
if not missing_libs:
 print("Perfetto! Il tuo ambiente è completissimo per la Cyber Security.")
else:
 print("Il tuo ambiente è operativo, ma mancano alcune librerie utili.")
 print("Puoi installare i pacchetti mancanti eseguendo:")
 print(f"pip install {' '.join(missing_libs)}")
...

...

Come verificare velocemente da Terminale (senza script)

Se vuoi solo controllare le librerie installate dal tuo terminale o prompt dei comandi, esegui:

```bash
pip list
```

Best Practice per l'ambiente di Cyber Security:
1. **Usa sempre un Ambiente Virtuale ('venv')**: Evita di installare pacchetti nella versione di Python di sistema per non creare conflitti.
    ```bash
    python -m venv cyber_env
    source cyber_env/bin/activate # Su Linux/macOS
    cyber_env\Scripts\activate   # Su Windows
    ```
2. **Strumenti esterni**: Ricorda che librerie come `python-nmap` richiedono che `Nmap` sia effettivamente installato sul sistema operativo, non basta la libreria Python.

Esegui lo script e fammi sapere se manca qualche pacchetto specifico o se stai configurando l'ambiente per un ambito particolare (es. *Web Pentesting, Forensics, Exploit Development, Network Analysis*) così posso darti consigli più mirati!

(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$
```

## Creazione del Security Assistant (security\_bot.py)

Superata la fase di verifica dell'ambiente, ho creato un secondo script, security\_bot.py, per realizzare un chatbot interattivo. Lo script:

- Carica le variabili d'ambiente e inizializza il client Gemini
- Definisce una system\_instruction che assegna al modello il ruolo di Hacker Etico e consulente esperto di Cyber Security, con il compito di individuare vulnerabilità e applicare best practice di difesa
- Crea una sessione di chat persistente con client.chats.create(), usando il modello gemini-3.6-flash
- Implementa un ciclo di conversazione (while True) che legge l'input dell'utente e termina con i comandi "exit" o "quit"



## Cross-Site Scripting (XSS)

Proseguendo la conversazione, l'assistente ha illustrato come prevenire attacchi XSS: da un lato l'uso di `textContent` al posto di `innerHTML` per inserire testo non fidato nel DOM, dall'altro l'utilizzo della libreria `DOMPurify` per sanificare eventuale HTML dinamico. Ha infine riassunto alcune regole d'oro anti-XSS, come evitare API pericolose (`innerHTML`, `document.write`, `eval`) e impostare un header `Content-Security-Policy`:

```
(venv ai)-(kali@kali) ~/Desktop/Epicode/python_ai
$ python security_bot.py

// SICURO: textContent tratta l'input PURAMENTE come testo, disattivando qualsiasi script
document.getElementById("welcome-message").textContent = "Benvenuto, " + username;
</script>
'''
*Risultato per l'utente: Vedrà a schermo il testo letterale '' senza che lo script venga eseguito.
'''

✅ CODICE SICURO 2: Se devi consentire tag HTML (Uso di 'DOMPurify')

Se l'utente deve poter inserire formattazione HTML (es. testo in grassetto ''), **non** usare 'innerHTML' grezzo. Devi prima sanitizzare l'input con una libreria affidabile come 'DOMPurify'.

'''html
<!-- Importa DOMPurify -->
<script src="https://cdn.jsdelivr.net/npm/dompurify@3.0.6/dist/purify.min.js"></script>

<script>
 const urlParams = new URLSearchParams(window.location.search);
 const userInput = urlParams.get('bio'); // es: "Ciao mondo <script>alert(1)</script>"

 // SICURO: DOMPurify rimuove i tag e gli attributi pericolosi (es. <script>, onerror=)
 const cleanHTML = DOMPurify.sanitize(userInput);

 // Ora puoi usare innerHTML in sicurezza
 document.getElementById("welcome-message").innerHTML = cleanHTML;
</script>
'''

🛡️ Regole d'oro anti-XSS:
1. **Evita API pericolose**: Riduci o elimina l'uso di 'innerHTML', 'outerHTML', 'document.write()' e 'eval()'.
2. **Usa 'textContent' di default**: Per inserire testo nei nodi DOM, usa sempre 'textContent'.
3. **Sanitizza se usi HTML**: Se devi rendere HTML dinamico, passa sempre i dati attraverso **DOMPurify**.
4. **Imposta l'header Content Security Policy (CSP)**: Un header HTTP come 'Content-Security-Policy: default-src 'self';' blocca l'esecuzione di script non autorizzati anche in presenza di XSS.
```

## Hashing sicuro delle password (bcrypt)

Ho poi chiesto all'assistente come implementare l'hashing sicuro delle password con `bcrypt`. La risposta ha coperto tre linguaggi diversi. In Python, tramite la libreria `bcrypt`, con le funzioni `hash_password` (che genera un salt con cost factor 12 e calcola l'hash) e `verify_password` (che confronta la password in modo sicuro contro i timing attack):

```
OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python security_bot.py

Tu: mostrami come implementare l'hashing sicuro delle password con bcrypt ?
Assistant: L'hashing sicuro delle password richiede l'uso di un algoritmo **adattivo** (lento per costruzione) come **Bcrypt**, che include automaticamente un **salt unico** per ogni
password per proteggere da attacchi Rainbow Table e Brute Force.

Ecco come implementare correttamente Bcrypt in **Python**, **Node.js** e **PHP**.

1. Python (Libreria 'bcrypt')
Prima installa il pacchetto: 'pip install bcrypt'

```python
import bcrypt

def hash_password(password: str) -> bytes:
    # Converti la stringa in byte
    password_bytes = password.encode('utf-8')
    # Genera un salt (cost factor raccomandato: 12)
    salt = bcrypt.gensalt(rounds=12)
    # Genera l'hash
    hashed = bcrypt.hashpw(password_bytes, salt)
    return hashed

def verify_password(password: str, hashed_password: bytes) -> bool:
    password_bytes = password.encode('utf-8')
    # Verifica sicura (protegge contro Timing Attacks)
    return bcrypt.checkpw(password_bytes, hashed_password)

# --- ESEMPIO D'USO ---
user_password = "MiaPasswordSicura123!"

# In fase di registrazione:
stored_hash = hash_password(user_password)

# In fase di login:
if verify_password("MiaPasswordSicura123!", stored_hash):
    print("Login effettuato!")
else:
    print("Credenziali errate.")
...

```

La stessa logica è stata poi mostrata in Node.js, con le funzioni asincrone `bcrypt.hash` e `bcrypt.compare` (visibile anche l'elenco dei file del progetto nel pannello laterale dell'editor):

```
0 .env
1 hello_gemini.py
2 requirements.txt
2 script.py
2 security_bot.py

--$ python security_bot.py
print("Login effettuato!")
else:
    print("Credenziali errate.")
...

---

### 2. JavaScript / Node.js (Libreria 'bcrypt')
Prima installa il pacchetto: 'npm install bcrypt'

```javascript
const bcrypt = require('bcrypt');
const SALT_ROUNDS = 12; // Cost factor raccomandato

// 1. Hash durante la registrazione (Async)
async function hashPassword(password) {
 const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);
 return hashedPassword;
}

// 2. Verifica durante il login (Async)
async function verifyPassword(password, hashedPassword) {
 const isMatch = await bcrypt.compare(password, hashedPassword);
 return isMatch;
}

// --- ESEMPIO D'USO ---
(async () => {
 const rawPassword = "MiaPasswordSicura123!";

 // Registrazione
 const dbHash = await hashPassword(rawPassword);

 // Login
 const isValid = await verifyPassword("MiaPasswordSicura123!", dbHash);
 console.log(isValid ? "Login OK" : "Login Fallito");
})();

```

Infine è stata mostrata l'implementazione nativa in PHP, tramite le funzioni `password_hash` e `password_verify` (che non richiedono librerie esterne), seguita da un riepilogo delle regole d'oro per la sicurezza delle password: cost factor 12, salt gestito automaticamente dalla libreria, funzioni di verifica dedicate a tempo costante e attenzione al limite di 72 byte gestito da `bcrypt`:

```

(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
$ python security_bot.py
...

3. PHP (Nativo)
PHP gestisce Bcrypt nativamente tramite le funzioni `password_hash` e `password_verify`. Non serve installare librerie esterne.

```php
<?php
// 1. Hash durante la registrazione
$password = "MiaPasswordSicura123!";

// PASSWORD_BCRYPT usa l'algoritmo Bcrypt. Cost raccomandato: 12
$hashedPassword = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);

// 2. Verifica durante il login
$userInput = "MiaPasswordSicura123!";

if (password_verify($userInput, $hashedPassword)) {
    echo "Login effettuato!";
} else {
    echo "Credenziali errate.";
}
?>
```
...

🏆 Regole d'oro per la Sicurezza delle Password:

1. **Cost Factor / Rounds**: Impostalo a **12**. Garantisce un ottimo equilibrio tra sicurezza (rallenta gli attaccanti) e prestazioni del server.
2. **Non generare il Salt manualmente**: Le librerie moderne di Bcrypt gestiscono il salt in automatico e lo incorporano nella stringa di hash finale.
3. **Usa funzioni di verifica dedicate**: Non confrontare mai le stringhe direttamente con `==` o `===`. Funzioni come `bcrypt.checkpw` o `password_verify` eseguono un confronto a **tempo costante** (prevenendo *Timing Attacks*).
4. **Lunghezza Input**: Bcrypt tronca l'input ai primi 72 byte. Se necessario gestire password più lunghe, considera l'uso di **Argon2id** (standard OWASP attuale).

```

## Conclusioni

Attraverso questa pratica ho imparato a:

- Configurare un ambiente Python per lavorare con l'API di Gemini e verificarne in autonomia la completezza rispetto alle librerie di Cyber Security
- Scrivere uno script base per inviare prompt e ricevere risposte da un LLM
- Costruire un chatbot interattivo con un ruolo definito tramite `system_instruction` e una sessione di chat persistente
- Approfondire, tramite il dialogo con l'assistente, tre vulnerabilità fondamentali della sicurezza applicativa: SQL Injection, XSS e la corretta gestione delle password con `bcrypt`

Questo esercizio ha unito le competenze di programmazione Python con l'uso pratico degli LLM come strumento di supporto e apprendimento in ambito Cyber Security.